Chapter 4

Java Collection Framework

Reference: H. Schildt, Java The Complete Reference (9th Ed.), Chapter 18, Oracle Press.

목차

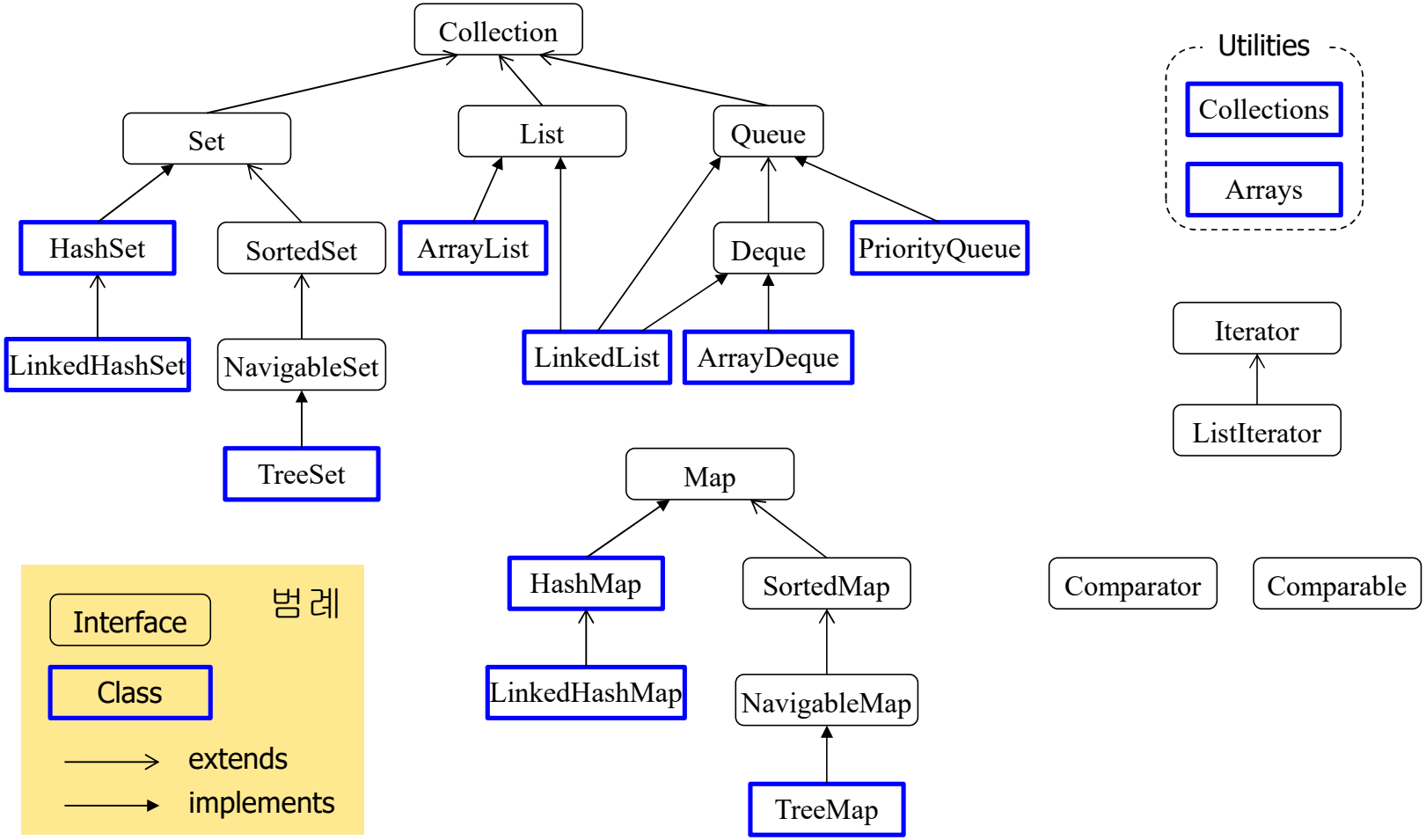
- 개요
- Collection Interfaces
- Collection Classes
- Iterator
- Map
- Comparators
- The Collection Algorithms
- Arrays
- The Legacy Classes

1. 개요

- Java Collection Framework 이란?
  - 다양한 자료구조(배열, 연결 리스트, 트리, 큐, ...)를 구현하는 Java API
  - 고성능 알고리즘들로 구현 (Collections, Arrays)
  - JDK 5.0부터 Generic type을 지원
    - 다양한 타입의 데이터들을 저장 가능

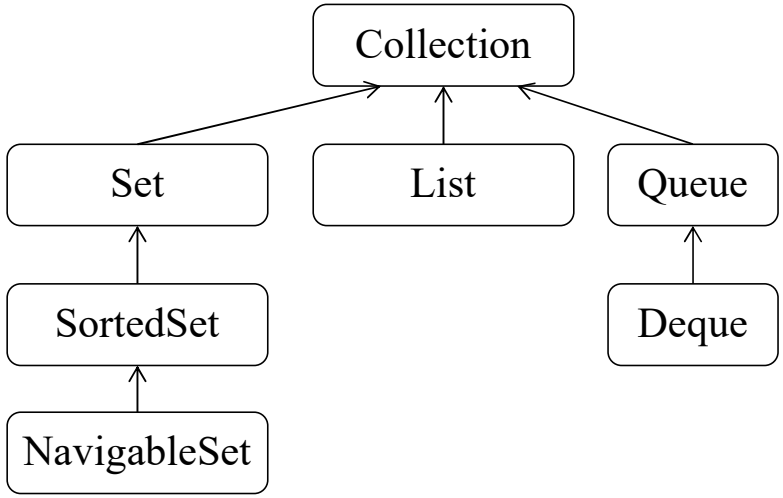
- Java Collection Framework의 구성 요소
  - Interface
    - Collection, List, Set, Queue, SortedSet, …
    - Map, SortedMap, …
    - Iterator, Enumeration, Spliterator, …
  - Class: 위의 인터페이스들을 구현한 다양한 클래스들

Class Hierarchy



2. Collection Interfaces

- Collection
  - Hierarchy의 root에 해당하며, 대부분의 collection 객체들에게 필요한 기능 정의
- List
  - 순서를 가지는 원소들의 모임
  - 중복 허용
- Set
  - 순서가 없고, 중복이 허용 안됨
- Queue
  - Head에서 삭제, tail에 추가 (FIFO)
- SortedSet
  - 오름차순으로 정렬된 Set
- NavigableSet
  - 주어진 값의 구간 검색 기능 제공
- Deque
  - Head와 tail에서 삽입/삭제가 모두 허용
  - Double-ended queue



2.1 Collection 인터페이스

- Collection framework의 기본 인터페이스
  - interface Collection<E>
    - 타입 E에 대한 generic interface로 정의
  - 주요 method들

Method	Description
boolean add(E obj)	원소 추가
boolean remove(Object obj)	원소 삭제
void clear( )	모든 원소 삭제
boolean isEmpty( )	공백 상태이면 true 반환
boolean contains(Object obj)	obj를 포함하고 있으면 true 반환
int size( )	원소의 개수 반환
Iterator<E> iterator( )	원소들을 방문하기 위한 iterator 객체 반환
<T> T[] toArray(T array[])	원소들을 배열에 복사하여 반환

2.2 List 인터페이스

- Collection의 sub-interface로 원소들의 순서 정의 및 중복 허용
  - interface List<E>
  - 주요 method들 (Collection에서 상속받은 내용은 제외)

Method	Description
void add(int index, E obj)	index 위치에 obj 추가
E set(int index, E obj)	index 위치의 원소를 obj로 대체 (old 값 반환)
E remove(int index)	index 위치의 원소를 삭제
E get(int index)	index 위치의 원소를 반환
int indexOf(Object obj)	obj가 처음으로 나오는 위치 반환 (Or, -1)
List<E> subList(int start, int end)	start ≤ 위치 < end 사이의 원소들로 구성된 list 반환
ListIterator<E> listIterator(int index)	index부터 list를 방문하기 위한 iterator 반환
default void sort(Comparator<? super E> comp)	comp를 이용하여 원소들을 정렬 (JDK 8부터 지원)

2.3 Set 인터페이스

- Collection의 sub-interface로 원소들의 순서가 없고, 중복 허용 안됨
  - interface Set<E>
  - Collection에서 상속받은 내용 외에 추가적인 method는 없음
  - 단, add() 의 동작 내용이 변경
    - boolean add(E obj) : 중복된 원소를 추가하려고 할 경우, false가 반환

2.4 SortedSet 인터페이스

- Set의 sub-interface로 원소들이 오름차순으로 정렬
  - interface SortedSet<E>
  - 주요 method들 (Set에서 상속받은 내용은 제외)

Method	Description
E first( )	첫 번째 원소(가장 작은 원소)를 반환
E last( )	마지막 원소를 반환
SortedSet<E> headSet(E end)	end보다 작은 (혹은 start보다 크거나 같은) 원소들로 구성된 SortedSet 반환 // 반환되는 set에는 원소의 reference만 포함
SortedSet<E> tailSet(E start)	
SortedSet<E> subSet(E start, E end)	start ≤ 크기 < end 사이의 원소로 구성
Comparator<? super E> comparator( )	SortedSet의 정렬 기준이 되는 comparator 객체를 반환. Natural ordering일 경우, null

2.5 NavigableSet 인터페이스

- SortedSet의 sub-interface로 주어진 값에 가장 가까운 원소를 기준으로 다양한 검색 기능들을 지원
  - interface NavigableSet<E>
  - 주요 method들 (SortedSet에서 상속받은 내용은 제외)

Method	Description
E ceiling(E obj), E floor(E obj)	obj ≤ e인 원소들 중에서 가장 작은 e를 반환
E higher(E obj), E lower(E obj)	obj < e인 원소들 중에서 가장 작은 e를 반환
E pollFirst( ), E pollLast( )	첫 번째 원소를 반환. 없으면 null 반환
NavigableSet<E> descendingSet( )	원소들을 역순으로 연결한 set의 view 반환
Iterator<E> descendingIterator( )	원소들을 역순으로 순회할 수 있는 iterator 반환
NavigableSet<E> headSet(E upper, boolean incl)	upper보다 작거나 같은(if incl == true) 원소들로 구성된 set의 view 반환

2.6 Queue 인터페이스

- Collection의 sub-interface로 head에서 원소 삭제, tail에서 원소 추가
  - interface Queue<E>
  - 주요 method들 (Collection에서 상속받은 내용은 제외)

Method	Description
boolean offer(E e) boolean add(E e)	e를 큐에 추가. Queue full이면, false return. (add: throw IllegalStateException)
E element( )	Head에 있는 원소 반환. QueueEmpty → throw NoSuchElementException
E peek( )	Head에 있는 원소 반환. QueueEmpty이면, null을 반환
E remove( )	Head에 있는 원소를 반환하고 삭제. QueueEmpty이면, NoSuchElementException
E poll( )	Head에 있는 원소를 반환하고 삭제. QueueEmpty이면, null 반환

	Throws exception	Returns special value
Insert	add(e)	offer(e)
Examine	element()	peek()
Remove	remove()	poll()

2.7 Deque 인터페이스

- Collection framework의 기본 인터페이스
  - interface Deque<E>
    - 타입 E에 대한 generic interface로 정의
  - 주요 method들

Method	Description
void addFirst(E obj), addLast(…)	Head (Tail)에 원소 추가, QueueFull이면 IllegalStateException 오류를 throw
boolean offerFirst(E obj), ..Last()	Head (Tail)에 원소 추가, 오류면 false 반환
E getFirst( ), getLast( )	처음 (마지막) 원소 반환. NoSuchElementException
E pollFirst( ), pollLast( )	처음 (마지막) 원소 반환하고 삭제. null
void push(E obj)	obj를 head에 추가. IllegalStateException
E pop( )	Head의 원소 반환하고 삭제. QueueEmpty 이면 NoSuchElementException 오류 throw

3. Collection Classes

Class	Description
AbstractCollection	Implements most of the **Collection** interface.
AbstractList	Extends **AbstractCollection** and implements most of the **List** interface.
AbstractSet	Extends **AbstractCollection** and implements most of the **Set** interface.
AbstractQueue	Extends **AbstractCollection** and implements parts of the **Queue** interface.
AbstractSequentialList	Extends **AbstractList** for use by a collection that uses sequential rather than random access of its elements.
ArrayList	Implements a dynamic array by extending **AbstractList**.
LinkedList	Implements a linked list by extending **AbstractSequentialList**.
ArrayDeque	Implements a dynamic double-ended queue by extending **AbstractCollection** and implementing the **Deque** interface.
HashSet	Extends **AbstractSet** for use with a hash table.
LinkedHashSet	Extends **HashSet** to allow insertion-order iterations.
PriorityQueue	Extends **AbstractQueue** to support a priority-based queue.
TreeSet	Implements a set stored in a tree. Extends **AbstractSet**.

3.1 ArrayList 클래스

- Object reference들을 원소로 갖는 동적 배열
  - AbstractList 클래스 확장, List 인터페이스 구현
  - class ArrayList<E>
  - 동적 배열 = Resizable-array
    - 원소가 추가될 때마다 배열의 크기가 증가
    - 원소가 삭제되면 배열의 크기도 축소

생성자	설명
ArrayList()	빈 array list를 생성(용량 = 10)
ArrayList(Collection<? extends E> c)	c에 포함된 원소들을 초기값으로 갖는 array list 생성
ArrayList(int capacity)	배열의 초기 크기를 capacity로 설정

ArrayList의 예

```
class ArrayListDemo {
    public static void main(String args[]) {
        ArrayList<String> al = new ArrayList<String>(); // 비어있는 array list 생성

        System.out.println("Initial size of al: " + al.size());

        // array list에 원소 추가
        al.add("C");    al.add("A");    al.add("E");    al.add("B");    al.add("D");
        al.add("F");    al.add(1, "A2");

        System.out.println("Size of al after additions: " + al.size());
        System.out.println("Contents of al: " + al);   // array list의 내용 출력. toString()

        al.remove("F");       al.remove(2);   // array list에서 원소 삭제

        System.out.println("Size of al after deletions: " + al.size());
        System.out.println("Contents of al: " + al);
    }
}
```

```
Initial size of al: 0
Size of al after additions: 7
Contents of al: [C, A2, A, E, B, D, F]
Size of al after deletions: 5
Contents of al: [C, A2, E, B, D]
```

ArrayList에서 배열 추출: toArray()

```
class ArrayListToArray {
    public static void main(String args[]) {

        ArrayList<Integer> al = new ArrayList<Integer>();   // Integer array list를 생성

        // array list에 원소 추가 ← autoboxing
        al.add(1);      al.add(2);      al.add(3);      al.add(4);

        System.out.println("Contents of al: " + al);

        Integer ia[] = new Integer[al.size()];  // 배열 준비.
        ia = al.toArray(ia);   // toArray() 함수를 이용하여 array list의 내용을 배열에 저장

        int sum = 0;
        for (int i : ia)    // Sum the array.
            sum += i;

        System.out.println("Sum is: " + sum);
    }
}
```

al.size()를 3이나 5로 변경할 경우?

```
Contents of al: [1, 2, 3, 4]
Sum is: 10
```

3.2 LinkedList 클래스

- 이중 연결 리스트를 이용하여 원소들을 연결
  - AbstractSequentialList 클래스 확장
  - List, Deque, 그리고 Queue 인터페이스 구현
  - class LinkedList<E>

생성자	설명
LinkedList()	빈 linked list를 생성
LinkedList(Collection<? extends E> c)	c에 포함된 원소들을 초기값으로 갖는 linked list 생성

LinkedList의 예

```
class LinkedListDemo {
    public static void main(String args[]) {
        LinkedList<String> ll = new LinkedList<String>();    // linked list 생성

        // linked list에 원소 추가: Collection과 List, Deque의 함수를 모두 사용
        ll.add("F");    ll.add("B");    ll.add("D");    ll.add("E");    ll.add("C");
        ll.addLast("Z");    ll.addFirst("A");    ll.add(1, "A2");
        System.out.println("Original contents of ll: " + ll);

        ll.remove("F");    ll.remove(2);    // linked list에서 원소 삭제: Collection, List
        System.out.println("Contents of ll after deletion: " + ll);

        ll.removeFirst();    ll.removeLast();  // 처음과 마지막 원소 삭제: Deque
        System.out.println("ll after deleting first and last: " + ll);

        String val = ll.get(2);    // Get and set a value ← 순차 검색!
        ll.set(2, val + " Changed");
        System.out.println("ll after change: " + ll);
    }
}
```

```
Original contents of ll: [A, A2, F, B, D, E, C, Z]
Contents of ll after deletion: [A, A2, D, E, C, Z]
ll after deleting first and last: [A2, D, E, C]
ll after change: [A2, D, E Changed, C]
```

ArrayList와 LinkedList의 비교 (1)

- ArrayList
  - 장점: get(index)와 같이 index를 통한 주소 계산이 빠르다.
  - 단점: add(i, e)의 경우, i부터 끝까지 원소들의 위치가 변경
    - 연속된 메모리 할당의 부담: 메모리 단편화 초래

- LinkedList
  - 장점: add(i, e)의 경우, 다른 원소들의 위치가 변경되지 않는다.
    - 메모리 사용이 효율적
  - 단점: 순차 액세스만 지원 → 주소를 통한 액세스 속도가 늦다.

성능 비교

```
public static void main(String[] args) {
    ArrayList<Integer> al = new ArrayList<>();
    LinkedList<Integer> ll = new LinkedList<>();
    Random rand = new Random();
    int pos;
    long start, end;

    start = System.currentTimeMillis();
    for (int i = 0; i < 100000; i++)
        al.add(0, i);
    end = System.currentTimeMillis();
    System.out.println("Duration = " + (end - start));

    al.clear();
    start = System.currentTimeMillis();
    for (int i = 0; i < 100000; i++) {
        pos = rand.nextInt(i+1);
        al.add(pos, i);
    }
    end = System.currentTimeMillis();
    System.out.println("Duration = " + (end - start));
```

637

299

```
start = System.currentTimeMillis();
for (int i = 0; i < 100000; i++)
    ll.add(0, i);
end = System.currentTimeMillis();
System.out.println("Duration = " + (end - start));

ll.clear();
start = System.currentTimeMillis();
for (int i = 0; i < 100000; i++) {
    pos = rand.nextInt(i+1);
    ll.add(pos, i);
}
end = System.currentTimeMillis();
System.out.println("Duration = " + (end - start));
}
```

5

14,639

ArrayList와 LinkedList의 비교 (2)

- Modern computer technology seems, by my testing, to give an enormous edge to arrays. **Elements of an array can be shifted and copied at insane speeds.** As a result arrays and ArrayList will, in most practical situations, outperform LinkedList on inserts and deletes, often dramatically. In other words, ArrayList will beat LinkedList at its own game.
- The downside of ArrayList is **it tends to hang onto memory space after deletions**, where LinkedList gives up space as it gives up entries.
- The *bigger* downside of arrays and ArrayList is **they fragment free memory and overwork the garbage collector**. As an **ArrayList expands**, it creates new, bigger arrays, copies the old array to the new one, and frees the old one. Memory fills with big contiguous chunks of free memory that are not big enough for the next allocation. Eventually there's no suitable space for that allocation. Even though 90% of memory is free, no individual piece is big enough to do the job. The GC will work frantically to move things around, but if it takes too long to rearrange the space, it will throw an OutOfMemoryException. If it doesn't give up, it can still slow your program way down.
- The worst of this problem can be **hard to predict**. Your program will run fine one time. Then, with a bit less memory available, with no warning, it slows or stops.
- **LinkedList uses small, dainty bits of memory and GC's love it**. It still runs fine when you're using 99% of your available memory.
- So in general, use ArrayList for smaller sets of data that are not likely to have most of their contents deleted, or when you have tight control over creation and growth. (For instance, creating one ArrayList that uses 90% of memory and using it without filling it for the duration of the program is fine. **Continually creating and freeing ArrayList instances that use 10% of memory will kill you.**) Otherwise, go with LinkedList (or a Map of some sort if you need random access). If you have very large collections (say over 100,000 elements), no concerns about the GC, and plan lots of inserts and deletes and no random access, run a few benchmarks to see what's fastest.

3.3 HashSet 클래스

- 해싱을 이용하여 Set의 원소들을 저장
  - AbstractSet 클래스 확장
  - Set 인터페이스 구현
  - class HashSet<E>

생성자	설명
HashSet()	빈 hash set를 생성 (초기 크기 = 16, fillRatio = 0.75)
HashSet(Collection<? extends E> c)	c에 포함된 원소들을 초기값으로 갖는 hash set 생성
HashSet(int capacity)	빈 hash set을 생성하되, 초기 set의 크기를 capacity로 설정 (default = 16)
HashSet(int capacity, float fillRatio)	빈 hash set을 생성하되, 초기 set의 capacity와 fillRatio를 설정

용량: 버킷의 수 → 원소 수가 capacity * fillRatio 보다 커질 경우, capacity를 대략 2배 정도로 늘리고 기존에 저장된 원소들은 rehash

HashSet의 예

```
class HashSetDemo {
   public static void main(String args[]) {

      HashSet<String> hs = new HashSet<String>();     // Create a hash set.

      // Add elements to the hash set.
      hs.add("Beta");
      hs.add("Alpha");
      hs.add("Eta");
      hs.add("Gamma");
      hs.add("Epsilon");
      hs.add("Omega");

      System.out.println(hs);
   }
}
```

[Gamma, Eta, Alpha, Epsilon, Omega, Beta]

3.4 LinkedHashSet 클래스

- 해시 테이블과 연결 리스트를 결합
  - 원소들이 입력된 순서를 연결 리스트로 구현
    - toString()에서 출력되는 순서
    - 앞의 예: [Beta, Alpha, Eta, Gamma, Epsilon, Omega]
  - HashSet을 확장

3.5 TreeSet 클래스

- Red-black 트리를 이용하여 오름차순으로 원소 저장
  - AbstractSet 클래스 확장
  - NavigableSet 인터페이스 구현
  - class TreeSet<E>

생성자	설명
TreeSet()	빈 tree set 생성 (정렬 기준 = natural order)
TreeSet(Collection<? extends E> c)	c에 포함된 원소들을 초기값으로 갖는 tree set 생성 (정렬 기준 = natural order)
TreeSet(Comparator<? super E> comp)	빈 tree set 생성(정렬 기준 = 입력 Comparator)
TreeSet(SortedSet<E> ss)	ss의 원소들을 초기값으로 갖는 tree set 생성 (정렬 기준 = ss와 동일)

TreeSet의 예

```
class TreeSetDemo {
    public static void main(String args[]) {

        TreeSet<String> ts = new TreeSet<String>();    // Create a tree set.

        // Add elements to the tree set.
        ts.add("C");
        ts.add("A");
        ts.add("B");
        ts.add("E");
        ts.add("F");
        ts.add("D");

        System.out.println(ts);
        System.out.println(ts.subSet("C", "F"));      // NavigableSet의 method.
    }
}
```

```
[A, B, C, D, E, F]
[C, D, E]
```

Set을 이용한 집합 연산

- s1과 s2가 Set이라고 가정
  - s1.containsAll(s2) // true if s2 $\subseteq$ s1
  - s1.addAll(s2) // s1 = s1 $\cup$ s2
  - s1.retainsAll(s2) // s1 = s1 $\cap$ s2
  - s1.removeAll(s2) // s1 = s1 – s2

  - 주의: 위 연산의 결과로 s1의 원래 내용이 변경
  - s1과 s2의 합집합을 별도로 저장하기 위해서는
    - s1을 다른 set s3에 복사 (s3의 생성자 이용)
    - s3.addAll(s2)

집합 연산의 예

```
public class SetTest1 {
    public static void main(String[] args) {
        Set<String> s1 = new HashSet<String>();
        Set<String> s2 = new HashSet<String>();

        s1.add("A"); s1.add("B"); s1.add("C");
        s2.add("A"); s2.add("D");

        Set<String> union = new HashSet<String>(s1);
        union.addAll(s2);

        Set<String> intersection = new HashSet<String>(s1);
        intersection.retainAll(s2);

        System.out.println("합집합 " + union);
        System.out.println("교집합 " + intersection);
    }
}
```

```
[A, B, C, D]
[A]
```

3.6 PriorityQueue 클래스

- Comparator를 기준으로 순서가 결정되는 큐 (Min heap or Max heap)
  - AbstractQueue 클래스 확장
  - Queue 인터페이스 구현
  - class PriorityQueue<E>

생성자	설명
PriorityQueue()	Default 큐 (기준 = natural order, 용량 = 11)
PriorityQueue(int capacity)	용량을 지정 (기준 = natural order)
PriorityQueue(Comparator<? super E> comp)	우선순위 기준을 지정 (용량 = 11)
PriorityQueue(int capacity, Comparator<? super E> comp)	용량과 우선순위 기준을 지정
PriorityQueue(Collection<? extends E> c)	c에 저장된 원소들을 초기값으로 저장
PriorityQueue(SortedSet<? extends E> c)	c에 저장된 원소들을 초기값으로 저장 (c와 동일한 기준 적용)

PriorityQueue의 예

```
public class PriorityQueueTest {
    public static void main(String[] args) {
        PriorityQueue<Integer> pq = new PriorityQueue<Integer>();    // natural ordering

        pq.add(30);
        pq.add(80);
        pq.add(20);
//      pq.add(10);
//      pq.add(5);

        System.out.println(pq);

        System.out.println("원소 삭제");
        while (!pq.isEmpty())
            System.out.println(pq.remove());
    }
}
```

```
[20, 80, 30]
원소 삭제
20
30
80
```

3.7 ArrayDeque 클래스

- 동적 배열을 이용하여 Deque를 구현
  - AbstractCollection 클래스 확장
  - Deque 인터페이스 구현
  - class ArrayDeque<E>

생성자	설명
ArrayDeque()	Default 디큐 (초기 용량 = 16)
ArrayDeque(int size)	초기 용량을 지정
ArrayDeque(Collection<? extends E> c)	c에 저장된 원소들을 초기값으로 저장

ArrayDeque의 예

```
class ArrayDequeDemo {
    public static void main(String args[]) {

        ArrayDeque<String> adq = new ArrayDeque<String>();  // Create an ArrayDeque

        // Use an ArrayDeque like a stack.
        adq.push("A");
        adq.push("B");
        adq.push("D");
        adq.push("E");
        adq.push("F");

        System.out.print("Popping the stack: ");

        while(adq.peek() != null)
            System.out.print(adq.pop() + " ");

        System.out.println();
    }
}
```

Popping the stack: F E D B A

4. Iterator

- Iterator
  - Collection에 포함된 원소들을 하나씩 액세스할 수 있는 기능을 제공하는 인터페이스
  - interface Iterator<E>
    - 단방향 순회 및 원소를 삭제하는 method 지원

- ListIterator
  - Iterator를 확장한 인터페이스
  - interface ListIterator<E>
  - 양방향 순회 및 원소의 삽입/수정/삭제 기능 지원

- For-Each 문으로도 Collection을 순회 가능

Iterator와 ListIterator가 지원하는 기능

	Method	Description
Iterator와 ListIterator 모두 지원	boolean hasNext( )	뒤에 원소가 있으면 true
	E next( )	뒤 원소를 반환 (NoSuchElementException)
	default void remove( )	현재 원소를 삭제
ListIterator 만 지원	boolean hasPrevious( )	앞에 원소가 있으면 true
	E previous( )	앞 원소를 반환 (NoSuchElementException)
	int nextIndex( )	뒤 원소의 번호를 반환
	int previousIndex()	앞 원소의 번호를 반환
	void add(E obj)	뒤 원소의 앞에 obj 추가
	void set(E obj)	현재 원소를 obj로 변경

Iterator와 ListIterator를 이용한 예

```
class IteratorDemo {
  public static void main(String args[]) {
    ArrayList<String> al = new ArrayList<String>();

    // Array list에 원소 추가
    al.add("C");  al.add("A");  al.add("E");
    al.add("B");  al.add("D");  al.add("F");

    // Iterator를 이용하여 al의 내용 출력
    System.out.print("Original contents of al: ");
    Iterator<String> itr = al.iterator();
    while (itr.hasNext()) {
      String element = itr.next();
      System.out.print(element + " ");
    }
    System.out.println();

  // 순회하면서 원소들을 수정
  ListIterator<String> litr = al.listIterator();
  while(litr.hasNext()) {
    String element = litr.next();
    litr.set(element + "+"); // 현재 위치의 원소 수정
  }
```

```
  System.out.print("Modified contents of al: ");
  itr = al.iterator();
  while(itr.hasNext()) {
    String element = itr.next();
    System.out.print(element + " ");
  }
  System.out.println();

  // 역순으로 순회하면서 출력
  System.out.print("Modified list backwards: ");
  while(litr.hasPrevious()) {  // litr은 현재 마지막에..
    String element = litr.previous();
    System.out.print(element + " ");
  }
  System.out.println();
 }
}
```

```
Original contents of al: C A E B D F
Modified contents of al: C+ A+ E+ B+ D+ F+
Modified list backwards: F+ D+ B+ E+ A+ C+
```

For-Each 문을 이용하여 Iterator 대체

- 순방향/순회 전용의 경우, For문 이용 가능

```
class ForEachDemo {
  public static void main(String args[]) {
    ArrayList<Integer> vals = new ArrayList<Integer>();
    vals.add(1);    vals.add(2);    vals.add(3);    vals.add(4);    vals.add(5);

    System.out.print("Original contents of vals: ");
    for (int v : vals)    // for loop을 이용해 원소 출력(순방향 순회만 지원 & 수정 불가)
      System.out.print(v + " ");
    System.out.println();

    int sum = 0;
    for (int v : vals)    // Now, sum the values by using a for loop.
      sum += v;

    System.out.println("Sum of values: " + sum);
  }
}
```

```
Original contents of vals: 1 2 3 4 5
Sum of values: 15
```

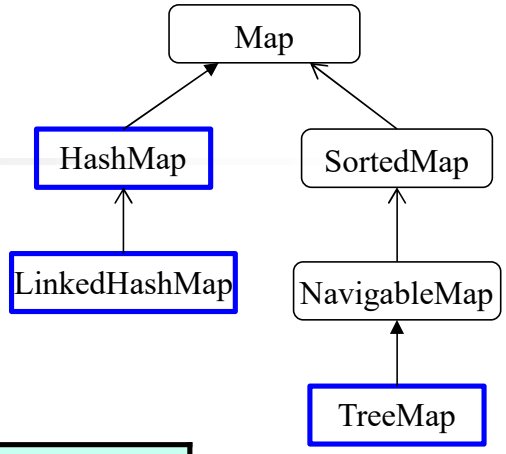
사용자 Class를 Collection에 저장

```
class Address {
  private String name;
  private String street;
  private String city;
  private String state;
  private String code;

  Address(String n, String s, String c,
          String st, String cd) {
    name = n;       street = s;
    city = c;       state = st;
    code = cd;
  }

  public String toString() {
    return name + "\n" + street + "\n" +
           city + " " + state + " " + code;
  }
}
```

```
class MailList {
  public static void main(String args[]) {
    LinkedList<Address> ml = new LinkedList<Address>();

    // Add elements to the linked list.
    ml.add(new Address("J.W. West", "11 Oak Ave",
                       "Urbana", "IL", "61801"));
    ml.add(new Address("Ralph Baker", "1142 Maple Lane",
                       "Mahome", "IL", "61853"));
    ml.add(new Address("Tom Carlton", "867 Elm St",
                       "Champaign", "IL", "61820"));

    // Display the mailing list.
    for(Address element : ml)
      System.out.println(element + "\n");

    System.out.println();
  }
}
```

Collection Framework을 이용하여,
프로그램의 생산성을 획기적으로 향상!

5. Map



- (키, 값)의 쌍을 저장 ← 주어진 키에 해당하는 값?
  - 키: 유일, 값: 중복이 가능
  - Iterable 인터페이스를 지원하지 않음 ← iterator() 메소드 구현 x
- Map Interfaces

Class	Description
Map	Maps unique keys to values.
Map.Entry	Describes an element (a key/value pair) in a map.
SortedMap	Extends **Map** so that the keys are maintained in ascending order.
NavigableMap	Extends **SortedMap** to handle the retrieval of entries based on closest-match.

- Map Classes

Class	Description
AbstractMap	Implements most of the **Map** interface.
HashMap	Extends **AbstractMap** to use a hash table.
LinkedHashMap	Extends **HashMap** to allow insertion-order iterations.
TreeMap	Extends **AbstractMap** to use a tree.

5.1 Map 인터페이스

- interface Map<K, V>
  - 키 K와 값 V의 쌍을 저장 (K는 유일할 것!)

Method	Description
V put(K k, V v)	(k, v) 쌍을 추가. k가 존재하면 덮어 쓰고, 이전 v 값 반환. 없을 경우, null 반환
V get(Object k)	k에 해당하는 값 반환. 없으면 null
V remove(Object k)	k에 해당하는 쌍 삭제
void clear( )	모든 쌍을 삭제
boolean isEmpty( )	Map이 비었으면 true 반환
boolean containsKey(Object k)	Map에 키 k가 있으면 true 반환(containsValue)
Set<K> keySet( )	Map에 포함되어 있는 키들의 Set 반환
Collection<V> values( )	Map에 포함되어 있는 값들의 Collection 반환
Set<Map.Entry<K, V>> entrySet( )	Map이 포함되어 있는 (키, 값) 쌍들의 Set 반환

5.2 SortedMap 인터페이스

- interface SortedMap<K, V>
  - Map을 확장 (키에 대한 오름차순으로 쌍들을 유지)

Method	Description
K firstKey( )	Map의 첫 번째 키를 반환
K lastKey( )	Map의 마지막 키를 반환
SortedMap<K, V> headMap(K end)	end보다 작은 키들로 구성된 Map 반환
SortedMap<K, V> tailMap(K start)	start보다 크거나 같은 키들로 구성된 Map 반환
SortedMap<K, V> subMap(K start, K end)	start <= 키 < end 인 키들로 구성된 Map 반환
Comparator<? super K> comparator( )	SortedMap의 comparator를 반환. Natural ordering이 사용될 경우, null을 반환

5.3 NavigableMap 인터페이스

- interface NavigableMap<K, V>
  - SortedMap을 확장 (주어진 키에 대한 구간 검색을 지원)

Method	Description
K ceilingKey(K obj), floorKey()	obj ≤ k인 키 중에서 가장 작은 키를 반환
K higherKey(K obj), lowerKey()	obj < k인 키 중에서 가장 작은 키를 반환
Map.Entry<K,V> pollFirstEntry( )	첫 번째 쌍을 반환. 없으면 null 반환
NavigableSet<K> descendingKeySet( )	키들을 역순으로 연결한 set의 view 반환
NavigableMap<K,V> descendingMap( )	호출 map의 역순인 map의 view 반환
Map.Entry<K,V> higherEntry(K obj)	obj < k인 키 중에서 가장 작은 키에 해당하는 쌍 반환
NavigableMap<K,V> headMap(K upper, boolean incl)	upper보다 작거나 같은(if incl == true) 키를 갖는 모든 쌍들로 구성된 map의 view 반환

5.4 Map.Entry 인터페이스

- interface Map.Entry<K, V>
  - (키, 값)의 쌍을 나타내며, Map의 내부 클래스로 구현

Method	Description
K getKey( )	이 map 항목(쌍)의 키를 반환
V getValue( )	이 map 항목(쌍)의 값을 반환
V setValue(V v)	이 map 항목(쌍)의 값을 v로 설정
boolean equals(Object obj)	obj의 키와 값이 이 map 항목과 동일하면 true 반환

두 개의 entry e1과 e2에 대해 아래 조건이 만족하면 e1.equals(e2) == true:

```
(e1.getKey()==null ? e2.getKey()==null : e1.getKey().equals(e2.getKey())) &&
(e1.getValue()==null ? e2.getValue()==null : e1.getValue().equals(e2.getValue()))
```

5.5 HashMap 클래스

- (키, 값)을 저장하는 해시 테이블
  - AbstractMap 클래스 확장
  - Map 인터페이스 구현: 키 값이 null인 경우도 허용.
  - class HashMap<K, V>

생성자	설명
HashMap()	빈 map 생성 (용량 = 16, fillRatio = 0.75)
HashMap(int capacity)	용량을 지정 (fillRatio = 0.75)
HashMap(int capacity, float fillRatio)	용량과 fillRatio를 지정
HashMap(Map<? extends K, ? extends V> m)	m에 저장된 (키, 값)의 쌍들을 초기값으로 저장

HashMap의 예

```
class HashMapDemo {
    public static void main(String args[]) {
        HashMap<String, Double> hm = new HashMap<String, Double>();

        hm.put("John Doe", new Double(3434.34));   // Map에 (키, 값) 쌍 추가
        hm.put("Tom Smith", new Double(123.22));
        hm.put("Jane Baker", new Double(1378.00));
        hm.put("Tod Hall", new Double(99.22));
        hm.put("Ralph Smith", new Double(-19.08));

         // Map에 저장된 쌍들을 모두 출력
        Set<Map.Entry<String, Double>> set = hm.entrySet();
        for (Map.Entry<String, Double> me : set)
            System.out.println(me.getKey() + ": " + me.getValue());

        // John Doe의 계정에 1000원을 추가
        double balance = hm.get("John Doe");
        hm.put("John Doe", balance + 1000);
        System.out.println("New balance: " +  hm.get("John Doe"));
}  }
```

```
Tod Hall: 99.22
John Doe: 3434.34
Ralph Smith: -19.08
Tom Smith: 123.22
Jane Baker: 1378.0
New balance: 4434.34
```

5.6 LinkedHashMap 클래스

- 해시 테이블과 연결 리스트를 결합
  - 원소들이 입력된 순서를 유지
  - HashMap을 확장
  - class LinkedHashMap<K, V>

LinkedHashMap의 예

```
public class LinkedHashMapDemo {
    public static void main(String[] args) {
        Map<String, Double> lhm = new LinkedHashMap<String, Double>();
        lhm.put("Apple", new Double(91.98));  lhm.put("Sony", new Double(16.76));
        lhm.put("Dell", new Double(30.47));    lhm.put("HP", new Double(33.91));

        System.out.println("Map contents : " + lhm);     // 입력된 순서대로 출력
        for (String key : lhm.keySet())
            System.out.println(key + ":\t" + lhm.get(key));  // toString()과 동일한 순서

        System.out.println("\nThe size of Map is : " + lhm.size());
        System.out.println("Is Map empty? : " + lhm.isEmpty());
        System.out.println("Map contains Sony as key? : " + lhm.containsKey("Sony"));

        System.out.println("\nRemove entry for Dell : " + lhm.remove("Dell"));
        System.out.println("Content of Map removing Dell: " + lhm);

        lhm.clear();
        System.out.println("Content of Map after clearing: " + lhm);
}  }
```

실행 결과

```
Map contents : {Apple=91.98, Sony=16.76, Dell=30.47, HP=33.91}
Apple:91.98
Sony:16.76
Dell:30.47
HP:33.91

The size of Map is : 4
Is Map empty? : false
Map contains Sony as key? : true

Remove entry for Dell : 30.47
Content of Map removing Dell: {Apple=91.98, Sony=16.76, HP=33.91}
Content of Map after clearing: {}
```

5.7 TreeMap 클래스

- 키의 정렬된 순서로 (키, 값)을 저장 ← Red-black tree
  - AbstractMap 클래스 확장
  - NavigableMap 인터페이스 구현
  - class TreeMap<K, V>

생성자	설명
TreeMap()	빈 tree map 생성 (정렬 기준 = natural order)
TreeMap(Comparator<? super K> comp)	comp를 정렬 기준으로 하는 빈 tree map 생성
TreeMap(Map<? extends K, ? extends V> m)	m에 저장된 (키, 값)의 쌍들을 초기값으로 저장 (정렬 기준 = natural order)
TreeMap(SortedMap<K, ? extends V> m)	m의 원소들을 초기값으로 갖는 tree map 생성 (정렬 기준 = m과 동일)

TreeMap의 예

```
class TreeMapDemo {
    public static void main(String args[]) {
        TreeMap<String, Double> tm = new TreeMap<String, Double>();

        tm.put("John Doe", new Double(3434.34));    // Map에 (키, 값) 쌍 추가
        tm.put("Tom Smith", new Double(123.22));
        tm.put("Jane Baker", new Double(1378.00));
        tm.put("Tod Hall", new Double(99.22));
        tm.put("Ralph Smith", new Double(-19.08));

         // Map에 저장된 쌍들을 모두 출력
        Set<Map.Entry<String, Double>> set = tm.entrySet();
        for(Map.Entry<String, Double> me : set)
            System.out.println(me.getKey() + ": " + me.getValue());

        // John Doe의 계정에 1000원을 추가
        double balance = tm.get("John Doe");
        tm.put("John Doe", balance + 1000);
        System.out.println("New balance: " +  tm.get("John Doe"));
}  }
```

```
Jane Baker: 1378.0
John Doe: 3434.34
Ralph Smith: -19.08
Tod Hall: 99.22
Tom Smith: 123.22
New balance: 4434.34
```

성능 비교

```
public static void main(String[] args) {
    HashMap<Double, Integer> hm = new HashMap<>();
    TreeMap<Double, Integer> tm = new TreeMap<>();

    ArrayList<Double> keys = new ArrayList<>();
    for (int i = 0; i < 1000000; i++)
        keys.add(Math.random());

    long start = System.currentTimeMillis();
    for (Double k : keys)
        hm.put(k, 1);
    end = System.currentTimeMillis();
   System.out.println("HashMap: 삽입 시간 = " + (end - start));

    start = System.currentTimeMillis();
    for (Double k : keys)
        tm.put(k, 1);
    end = System.currentTimeMillis();
   System.out.println("TreeMap: 삽입 시간 = " + (end - start));
```

151

804

```
    start = System.currentTimeMillis();
    for (Double k : keys)
        hm.get(k);
    end = System.currentTimeMillis();
    System.out.println("HashMap: 검색 시간 = " +
                                (end - start));

    start = System.currentTimeMillis();
    for (Double k : keys)
        tm.get(k);
    end = System.currentTimeMillis();
    System.out.println("TreeMap: 검색 시간 = " +
                            (end - start));
}
```

80

752

6. Comparator

- Comparator란?
  - 원소간의 크기 비교를 정의하는 인터페이스
  - Natural ordering을 변경할 경우, 구현하여야 함
  - interface Comparator<T>

Method	Description
int compare(T a, T b)	(a > b)면 양수, 같으면 0, (a < b)면 음수를 반환
default Comparator<T> reversed( )	이 comparator의 순서를 반대로 하는 comparator 반환 (JDK 8부터 지원)
default Comparator<T> thenComparing(Comparator<? super T> other)	이 comparator가 0을 반환할 경우, 두 번째로 검사할 comparator 설정 (JDK 8부터 지원)

Comparator의 예(1)

```
class MyComp implements Comparator<String> {
    public int compare(String aStr, String bStr) {
        return bStr.compareTo(aStr);     // Reverse the comparison.
    }
}

public class ComparatorDemo1 {
    public static void main(String args[]) {
        TreeSet<String> ts = new TreeSet<String>(new MyComp());

        // Add elements to the tree set.
        ts.add("C");   ts.add("A");   ts.add("B");   ts.add("E");   ts.add("F");   ts.add("D");

        System.out.println(ts);
    }
}
```

[F, E, D, C, B, A]

Comparator의 예(2)

```
class MyComp implements Comparator<String> {
    public int compare(String aStr, String bStr) {
        return aStr.compareTo(bStr);     // natural ordering
    }
}

public class ComparatorDemo2 {
    public static void main(String args[]) {
        MyComp mc = new MyComp();
        TreeSet<String> ts = new TreeSet<String>(mc.reversed());

        // Add elements to the tree set.
        ts.add("C");   ts.add("A");   ts.add("B");   ts.add("E");   ts.add("F");   ts.add("D");

        System.out.println(ts);
    }
}
```

[F, E, D, C, B, A]

TreeMap의 예

```
class TreeMapDemo {
    public static void main(String args[]) {
        TreeMap<String, Double> tm = new TreeMap<String, Double>();

        tm.put("John Doe", new Double(3434.34));    // Map에 (키, 값) 쌍 추가
        tm.put("Tom Smith", new Double(123.22));
        tm.put("Jane Baker", new Double(1378.00));
        tm.put("Tod Hall", new Double(99.22));
        tm.put("Ralph Smith", new Double(-19.08));

         // Map에 저장된 쌍들을 모두 출력
        Set<Map.Entry<String, Double>> set = tm.entrySet();
        for(Map.Entry<String, Double> me : set)
            System.out.println(me.getKey() + ": " + me.getValue());

        // John Doe의 계정에 1000원을 추가
        double balance = tm.get("John Doe");
        tm.put("John Doe", balance + 1000);
        System.out.println("New balance: " +  tm.get("John Doe"));
}  }
```

```
Jane Baker: 1378.0
John Doe: 3434.34
Ralph Smith: -19.08
Tod Hall: 99.22
Tom Smith: 123.22
New balance: 4434.34
```

예(3) – Last Name으로 비교

```
class TComp implements Comparator<String> {
    public int compare(String aStr, String bStr) {
            int i, j, k;

            i = aStr.lastIndexOf(' ');      // Last name의 앞 공백 문자의 위치
            j = bStr.lastIndexOf(' ');

            k = aStr.substring(i).compareToIgnoreCase(bStr.substring(j));
            return (k == 0) ? aStr.compareToIgnoreCase(bStr) : k;
    }
}

class TreeMapDemo2 {
    public static void main(String args[]) {
        TreeMap<String, Double> tm =
            new TreeMap<String, Double>(new TComp());
```

```
Jane Baker: 1378.0
John Doe: 3434.34
Tod Hall: 99.22
Ralph Smith: -19.08
Tom Smith: 123.22
New balance: 4434.34
```

예(4) – Last Name으로 비교

```
class CompLastNames implements Comparator<String> {
    public int compare(String aStr, String bStr) {
        int i = aStr.lastIndexOf(' ');
        int j = bStr.lastIndexOf(' ');
        return aStr.substring(i).compareToIgnoreCase(bStr.substring(j));
}  }

class CompThenByFirstName implements Comparator<String> {
    public int compare(String aStr, String bStr) {
        return aStr.compareToIgnoreCase(bStr);
}  }

class TreeMapDemo2A {
    public static void main(String args[]) {
        CompLastNames compLN = new CompLastNames();
        Comparator<String> compLastThenFirst =
                    compLN.thenComparing(new CompThenByFirstName());
        TreeMap<String, Double> tm =
                    new TreeMap<String, Double>(compLastThenFirst);
```

```
Jane Baker: 1378.0
John Doe: 3434.34
Tod Hall: 99.22
Ralph Smith: -19.08
Tom Smith: 123.22
New balance: 4434.34
```

7. The Collection Algorithms

- Collection이나 Map에 적용가능한 알고리즘들을 제공
  - Collections 클래스의 static method로 정의

Method	Description
static <T extends Comparable<? super T>> void sort(List<T> list)	리스트를 정렬 (natural ordering)
static <T> void sort(List<T> list, Comparator<? super T> comp)	리스트를 정렬 (Comparator 이용)
static void shuffle(List<T> list)	리스트를 섞기
static <T> int binarySearch(List<? extends Comparable<? super T>> list, T value)	정렬된 리스트에서 value를 이진 검색
static <T extends Object & Comparable<? super T>> T min(Collection<? extends T> c)	Collection에서 가장 작은 원소를 반환 (max() 도 당연히 존재...)
static void swap(List<?> list, int idx1, int idx2)	리스트에서 idx1의 원소와 idx2의 원소를 교환
static void reverse(List<T> list)	리스트의 원소들의 순서를 역전

Collections 함수 (계속)

Method	Description
static <T> void copy(List<? super T> list1, List<? extends T> list2)	list2의 원소들을 list1에 복사
static boolean disjoint(Collection<?> a, Collection<?> b)	a와 b에 공통된 원소가 없으면 true, 하나라도 있으면 false 반환
static void rotate(List<T> list, int n)	리스트의 원소들을 n만큼 shift right
static int frequency(Collection<?> c, object obj)	c에서 obj가 나타나는 빈도수 반환
static <T> Comparator<T> reverseOrder(Comparator<T> comp)	comp에서 정의된 순서를 역전시키는 comparator 반환
static <K, V> Map<K, V> checkedMap((Map<K, V> c, Class<K> k, Class<V> v)	Map의 type safe한 뷰를 반환(기존 값 변경이나 잘못된 타입 입력시 exception)
static <T> List<T> synchronizedList(List<T> list)	List의 thread safe한 뷰를 반환 (synchronized 문이 없을 경우 오류)
static <T> Set<T> unmodifiableSet(Set<? extends T> s)	Set의 수정 불가능한 뷰를 반환

Collections의 예

```
public class AlgorithmsDemo {
    public static void main(String args[]) {
        LinkedList<String> ll = new LinkedList<String>();
        ll.add("C");  ll.add("A");  ll.add("E");  ll.add("D");  ll.add("A");

        Comparator<String> r = Collections.reverseOrder();  // 역순의 comparator 생성
        Collections.sort(ll, r);     // Sort list by using the comparator
        System.out.println("List sorted in reverse: " + ll);

        Collections.shuffle(ll);     // Shuffle list
        System.out.println("List shuffled: " + ll);

        System.out.println("Minimum: " + Collections.min(ll));
        System.out.println("Maximum: " + Collections.max(ll));
        System.out.println("A의 빈도수: " + Collections.frequency(ll, "A"));
    }
}
```

```
List sorted in reverse: [E, D, C, A, A]
List shuffled: [A, D, E, A, C]
Minimum: A
Maximum: E
A의 빈도수: 2
```

8. Arrays

- 배열에 적용가능한 알고리즘들을 제공
  - Arrays 클래스의 static method들로 구현

Method	Description
static void sort(int array[])	int 배열을 정렬
static void sort(Object array[])	객체 배열을 정렬
static int binarySearch(int array[], int value)	정렬된 배열에서 이진검색으로 값 찾기
static void fill(int array[], int value)	배열의 모든 원소를 value로 설정 (start와 end를 추가할 수도 있음)
static int[] copyOf(int[] source, int len)	배열의 내용을 복사한 새로운 배열 반환
static int[] copyOfRange(int[] source, int start, int end)	start부터 end-1까지의 원소들을 복사한 새로운 배열 반환
static boolean equals(int array1[], int array2 [])	두 배열이 같으면 true, 다르면 false

Arrays의 예

```
class ArraysDemo {
    public static void main(String args[]) {
        int array[] = new int[8];
        for(int i = 0; i < 8; i++)
            array[i] = -3 * i;      // 배열 초기화

        // Display, sort, and display the array.
        System.out.print("Original contents: ");
        display(array);
        Arrays.sort(array);
        System.out.print("Sorted: ");
        display(array);

        // Fill and display the array.
        Arrays.fill(array, 2, 6, -1);
        System.out.print("After fill(): ");
        display(array);

        // Sort and display the array.
        Arrays.sort(array);
        System.out.print("After sorting again: ");
        display(array);

        // Binary search for -3.
        System.out.print("-3의 위치: ");
        int index = Arrays.binarySearch(array, -3);

        System.out.println(index);
    }

    static void display(int array[]) {
        for(int i: array)
            System.out.print(i + " ");

        System.out.println();
    }  // Arrays.toString(array)로 대체 가능
}
```

```
Original contents: 0 -3 -6 -9 -12 -15 -18 -21
Sorted: -21 -18 -15 -12 -9 -6 -3 0
After fill(): -21 -18 -1 -1 -1 -1 -3 0
After sorting again: -21 -18 -3 -1 -1 -1 -1 0
-3의 위치: 2
```

Arrays.sort()와 Collections.sort() 비교

- Arrays.sort()의 정렬 알고리즘
  - Primitive data type의 경우: Dual-Pivot Quicksort
  - Object type의 경우: Mergesort

- Collections.sort()의 정렬 알고리즘
  - Mergesort

- 질문: Java 개발자들은 왜 두 가지의 정렬 알고리즘을 구현하였을까?

Collections.sort() 의 소스 코드

```
public static <T> void sort(List<T> list, Comparator<? super T> c) {
    list.sort(c);
}
```

java.util.Collections#sort

```
default void sort(Comparator<? super E> c) {
    Object[] a = this.toArray();
    Arrays.sort(a, (Comparator) c);
    ListIterator<E> i = this.listIterator();
    for (Object e : a) {
        i.next();
        i.set((E) e);
    }
}
```

java.util.List#sort

9. The Legacy Classes

- Collection Framework이 없던 초기 Java부터 사용
  - Legacy 클래스는 Collection Framework의 클래스로 대체 가능
  - 그럼, 왜 사용하지?
  - 모든 legacy 클래스들은 synchronized 기능 제공

Legacy class	Description
Vector	동적 배열을 구현하며 ArrayList와 유사 Collections에서 지원되는 다양한 기능들을 자체적으로 제공
Stack	Vector의 서브 클래스로 last-in-first-out 기능을 지원 ArrayDeque로 대체 가능
Dictionary	Map과 유사하게 (키, 값)의 쌍을 저장하는 기능을 제공하는 추상 클래스. Map으로 대체 가능
HashTable	Dictionary를 구현하였으나, 최근 JDK에서는 Map을 구현하도록 확장. synchronized HashMap으로 사용되고 있음
Properties	HashTable의 서브 클래스로 (String, String) 쌍들을 저장하도록 특화. 시스템의 환경 변수 등을 저장할 때 사용